

ncps architecture

NCPS Prototype Architecture

Backend, Pipeline, and Data Flow Specification

1. System Overview

The system is an event-driven, stateful backend that continuously updates:

- user state
- post state
- graph structure

based on incoming events such as:

- post creation
- user interactions
- location updates

The architecture follows a stream-processing + state-store model, where:

Every event incrementally updates the global system state.

2. High-Level Architecture

[Client (Web/App)] ↓ [API Gateway] ↓ [Event Ingestion Layer] ↓ [Stream Processor] ↓ [Core Computation Modules] ↓ [State Store (DB + Cache)] ↓ [Decision Engine] ↓ [Notification / Feed System]

3. Core Components

3.1 API Gateway

Responsibilities

- Accept all incoming requests
- Authenticate users
- Normalize request format

Endpoints

POST /post/create POST /post/vote POST /user/location GET /feed

3.2 Event Ingestion Layer

Purpose

Convert API requests into structured events.

Event Format

```
{ "event_id": "...", "type": "VOTE | POST | LOCATION", "user_id": "...", "post_id": "...", "timestamp": "...", "payload":  
{ ... } }
```

Output

- Push events into event queue / stream
-

3.3 Stream Processor

Role

Processes events sequentially and triggers updates.

Processing Model

for each event e: route_to_handler(e.type)

4. Core Computation Modules

These are the heart of the system.

4.1 User State Module

Maintains

UserState: α_i β_i R_i $Conf_i$ R_i^* Exp_i $Anom_i$ T_i L_i w_i

Update Logic

On Vote Resolution

if vote_correct: $\alpha_i \ += \exp(-\lambda_r * \Delta_t)$ else: $\beta_i \ += \exp(-\lambda_r * \Delta_t)$

Compute Reliability

$R_i = \alpha_i / (\alpha_i + \beta_i)$ $Conf_i = 1 - \exp(-k * (\alpha_i + \beta_i))$ $R_{i_star} = R_i * Conf_i$

Compute Experience

$E_i = \sum(\exp(-\lambda_E * \delta_t))$ $Exp_i = \log(1 + E_i) / \log(1 + E_{max})$

Compute Final Weight

$w_i = T_i * (1 - Anom_i) * Exp_i$

4.2 Post State Module

Maintains

PostState: interactions[] N_j S_j C_{Bayes} C_{final} Var_j $Burst_j$ U_j r_j

Update on New Interaction

add interaction to list weight = $w_i * \exp(-\lambda * \delta_t)$ if vote == +1: $S_{plus} += weight$ else: $S_{minus} += weight$ $N_j += weight$

Compute Credibility

$C_{Bayes} = (S_{plus} + \alpha_0) / (S_{plus} + S_{minus} + \alpha_0 + \beta_0)$ $C_{final} = (1 - \alpha - \gamma) * C_{Bayes} + \alpha * C_{ML} + \gamma * C_{memory}$

Compute Variance

$Var_j = \sum(w_i * (s_k - C_{final})^2) / \sum(w_i)$

4.3 Graph Module (Adjacency Matrix A)

Data Structure

Graph: adjacency_list[user_i] = list of (user_j, weight)

Edge Update Logic

if users i and j interacted on same post: update: agreement_score time_similarity frequency_score $A_{ij} = w_1 * \text{agree} + w_2 * \text{time} + w_3 * \text{freq}$

Normalization

for each user i : $\text{sum_weights} = \sum(A_{ij})$ for each neighbor j : $A_{\text{norm_ij}} = A_{ij} / \text{sum_weights}$

Trust Propagation

$T_{\text{next}} = \lambda_g * A_{\text{norm}} * T + (1 - \lambda_g) * R_{\text{star}}$

4.4 Spatial Module

Maintains

LocationState: lat lon history[] L_i

Compute Location Confidence

$L_i = w_1 S_1 + w_2 S_2 + w_3 S_3 + w_4 S_4 + w_5 S_5$

Compute Proximity

$\text{distance} = \text{haversine}(\text{user}, \text{post})$ $\text{Prox} = L_i * \exp(-\text{distance}^2 / (2 * \sigma_p^2))$

4.5 Urgency Module

Compute Components

$K_j = \sum(\phi(\text{word})) / \text{total_words}$ $\text{Cat}_j = \text{classifier_output}$ $\text{rate}_j = N_j / \Delta t$ $V_j = 1 - \exp(-\text{rate}_j / \text{baseline})$

Final Urgency

$U_j = 1 - \exp(-(b_1 K_j + b_2 \text{Cat}_j + b_3 V_j))$

5. Decision Engine

5.1 Propagation Decision

```
if ( C_j >= theta AND N_j >= N_min AND Var_j <= sigma^2 AND Stable AND L_bar >= L_min ):
    expand_radius(j)
```

5.2 Alert Decision

```
if ( Prox(u,j) >= tau_p AND C_j * U_j >= theta_alert AND Var_j <= sigma^2 AND not RateLimited(u) ):
    send_alert(u, j)
```

6. Data Storage Design

6.1 User Table

user_id alpha beta R_star Exp Anom T L

6.2 Post Table

post_id C_final Var N U radius timestamp

6.3 Interaction Table

interaction_id user_id post_id vote timestamp weight

6.4 Graph Storage

user_id neighbor_id edge_weight last_updated

7. Data Flow Summary

End-to-End Flow

User Action ↓ API Gateway ↓ Event Created ↓ Stream Processor ↓ User State Update ↓ Post State Update ↓
Graph Update ↓ Spatial Update ↓ Urgency Update ↓ Decision Engine ↓ Feed / Alerts Updated

8. Real-Time vs Async Processing

Real-Time

- vote handling
- credibility update
- alert decision

Asynchronous

- ML inference
- trust graph propagation
- memory similarity

9. Key System Properties

Deterministic Core

All decisions are based on:

- explicit formulas
- stored state

Incremental Updates

No recomputation from scratch:

- all updates are delta-based

Bounded Outputs

Every major variable:

- lies in $[0,1]$
- prevents instability

10. Minimal Viable Prototype Stack

Backend

- Python (FastAPI)
 - Redis (cache)
 - PostgreSQL (state store)
-

Streaming

- Kafka
-

ML

- lightweight embedding service
 - optional batch processing
-

11. Final Interpretation

This architecture ensures that:

- every event updates system belief
- trust flows through users and relationships
- propagation is controlled, not automatic
- alerts are selective and meaningful